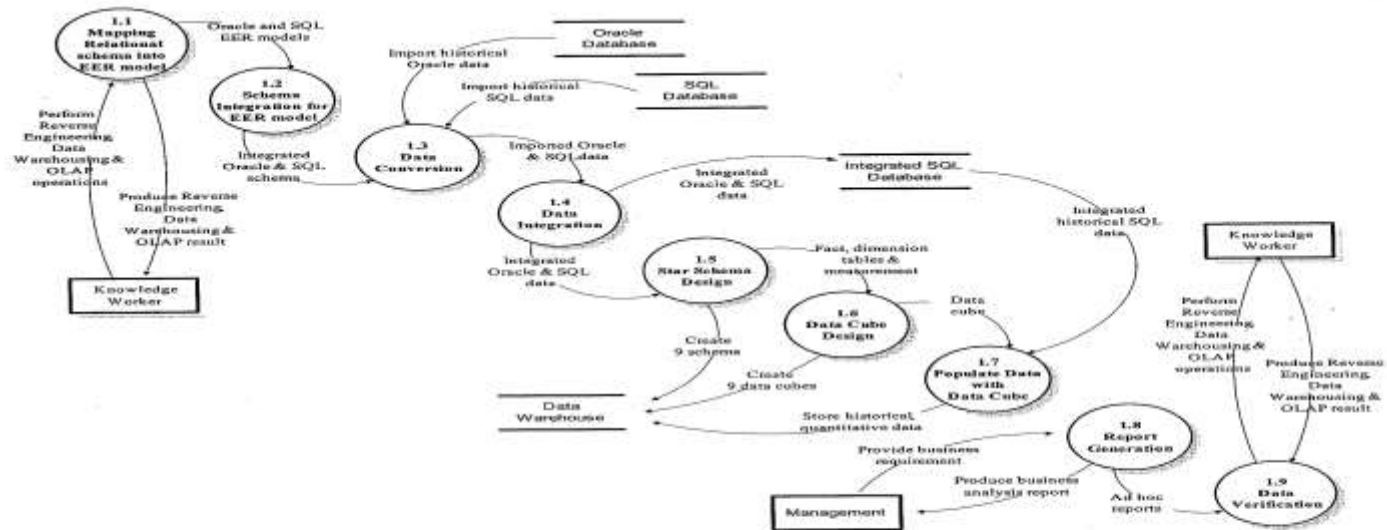


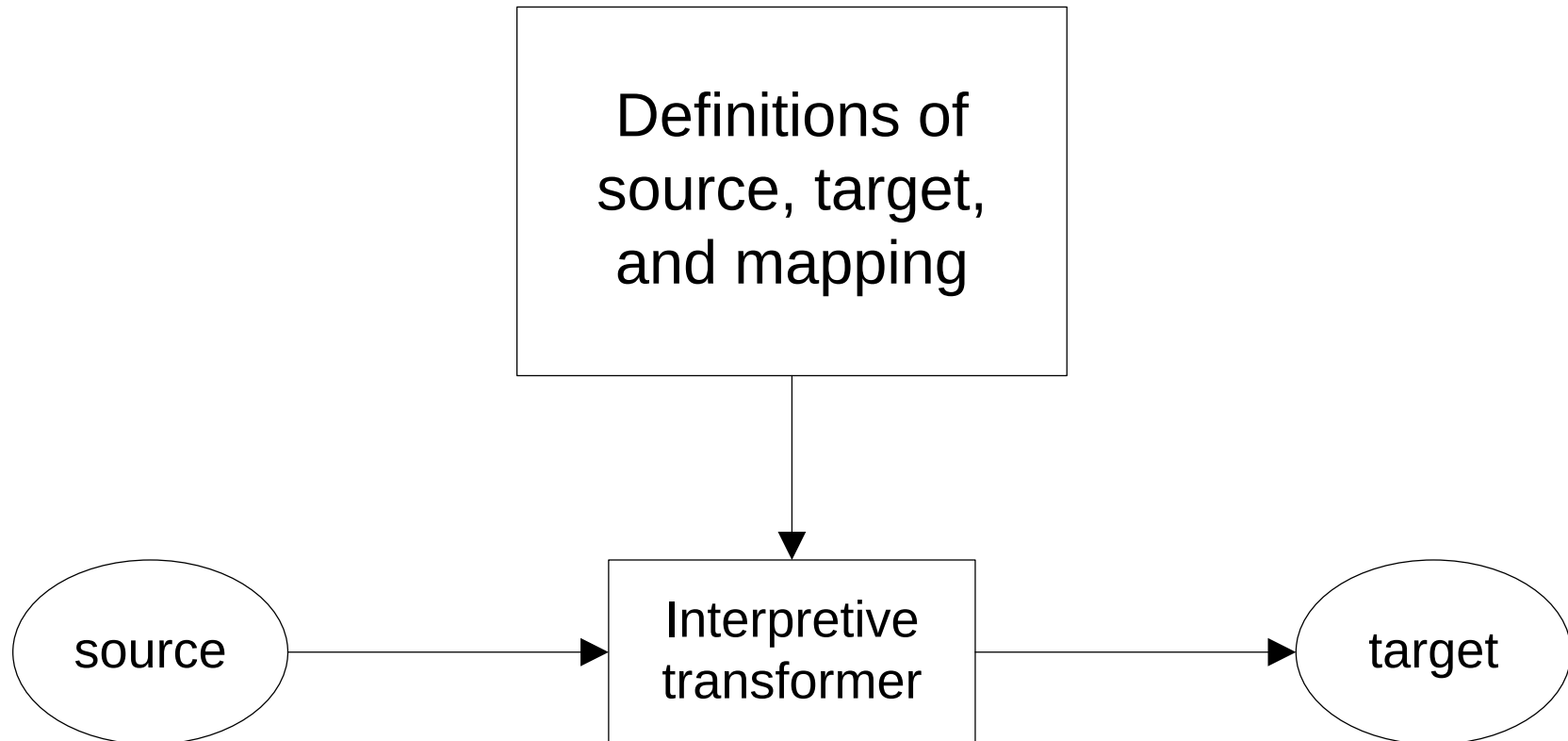
Methodology for Data warehousing with OLAP



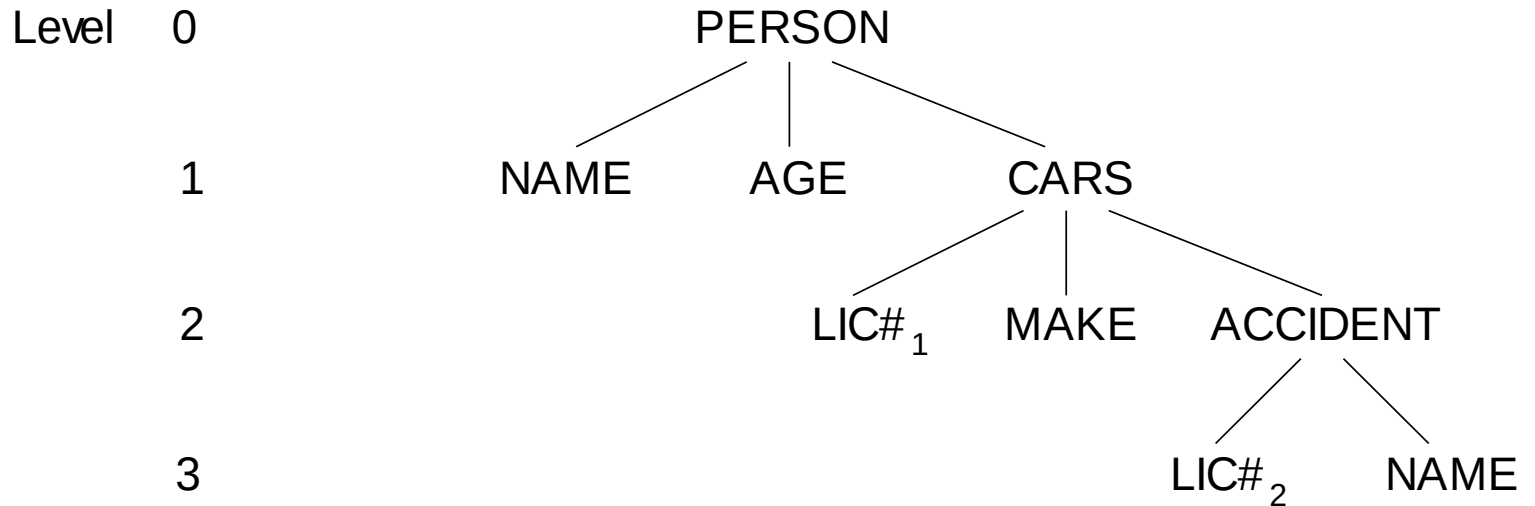
Data conversion: Customized Program Approach

A common approach to data conversion is to develop customized programs to transfer data from one environment to another. However, the customized program approach is extremely expensive because it requires a different program to be written for each M source files and N target. Furthermore, these programs are used only once. As a result, totally depending on customized program for data conversion is unmanageable, too costly and time consuming.

Interpretive Transformer approach of data conversion



For instance, the following Cobol structure



can be expressed in using these SDDL statements:

Data Structure

PERSON <NAME, AGE>

Instance

CARS <LIC#1, MAKE>

N-tuples

ACCIDENTS <LIC#2, NAME>

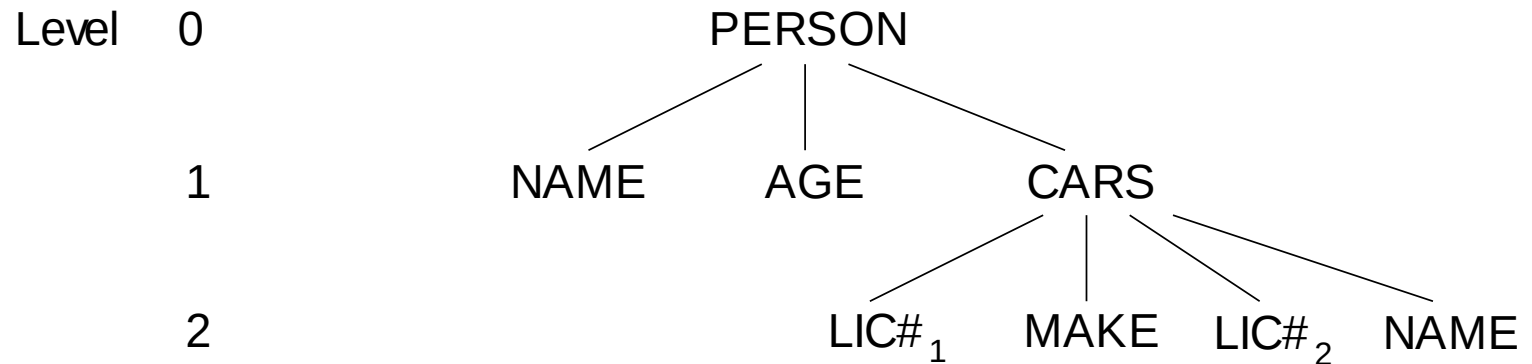
Relationship

PERSON-CAR <NAME, LIC#1>

N-tuples

CAR-ACCIDENT <LIC#1, LIC#2>

To translate the above three levels to the following two levels data structure:



the TDL statements are

FORM NAME FROM NAME

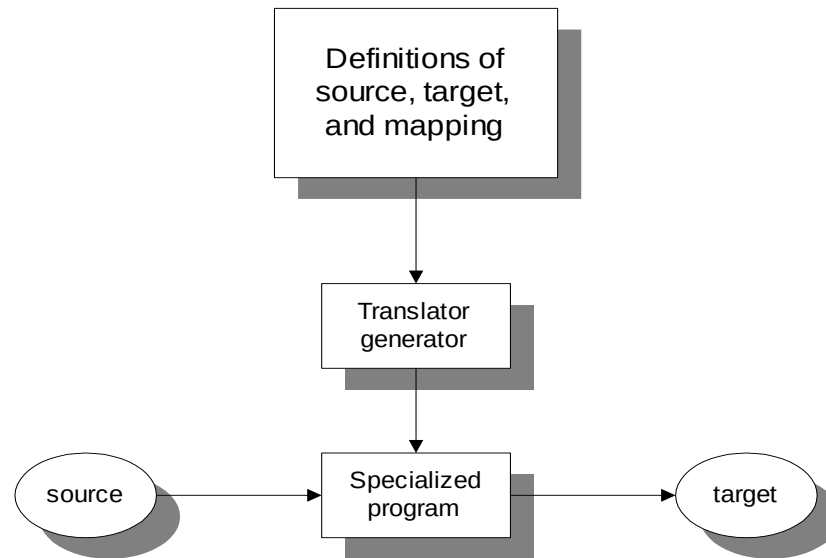
FORM LIC#1 FROM LIC#1

:

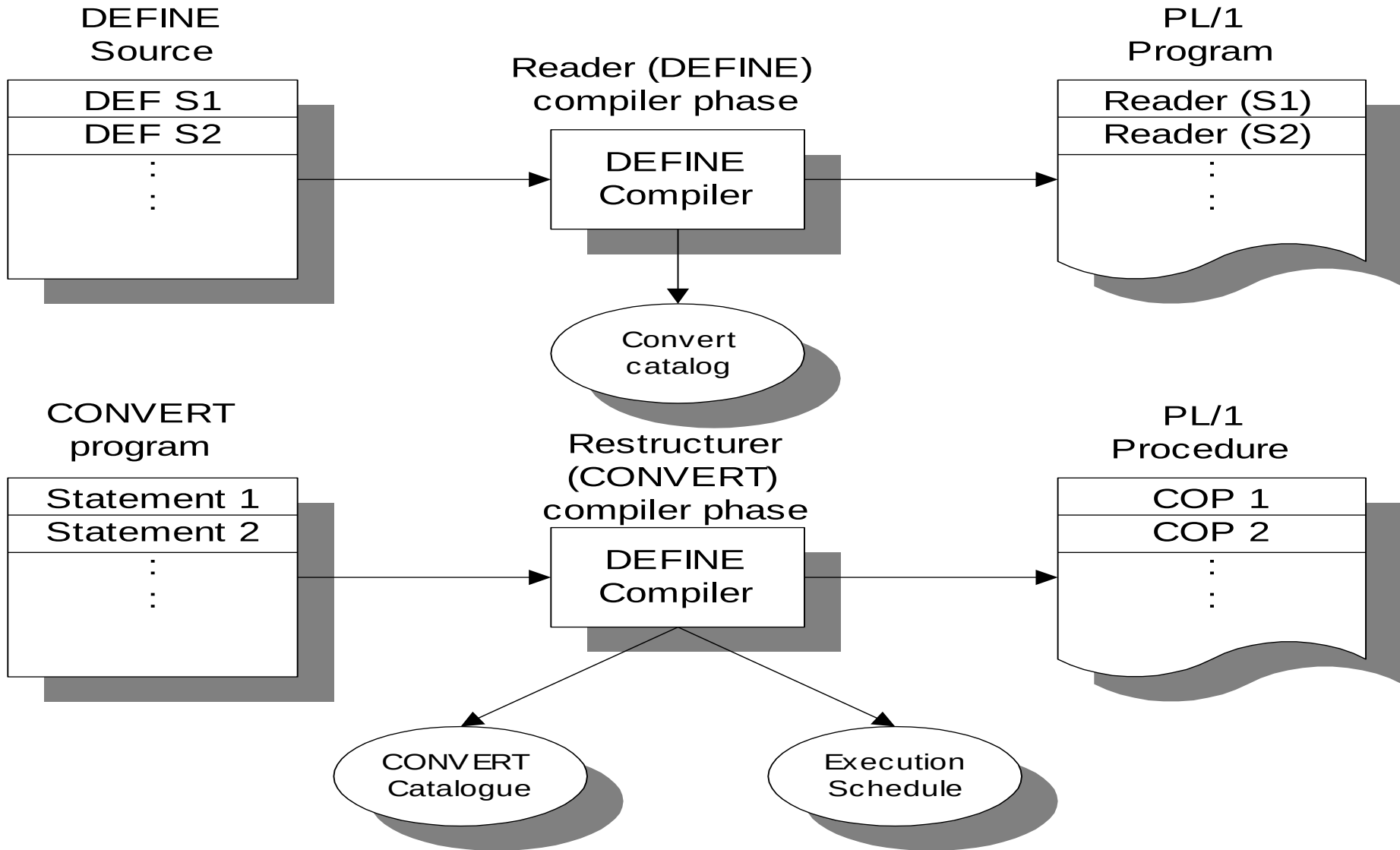
FORM PERSON IF PERSON

FORM CARS IF CAR AND ACCIDENT

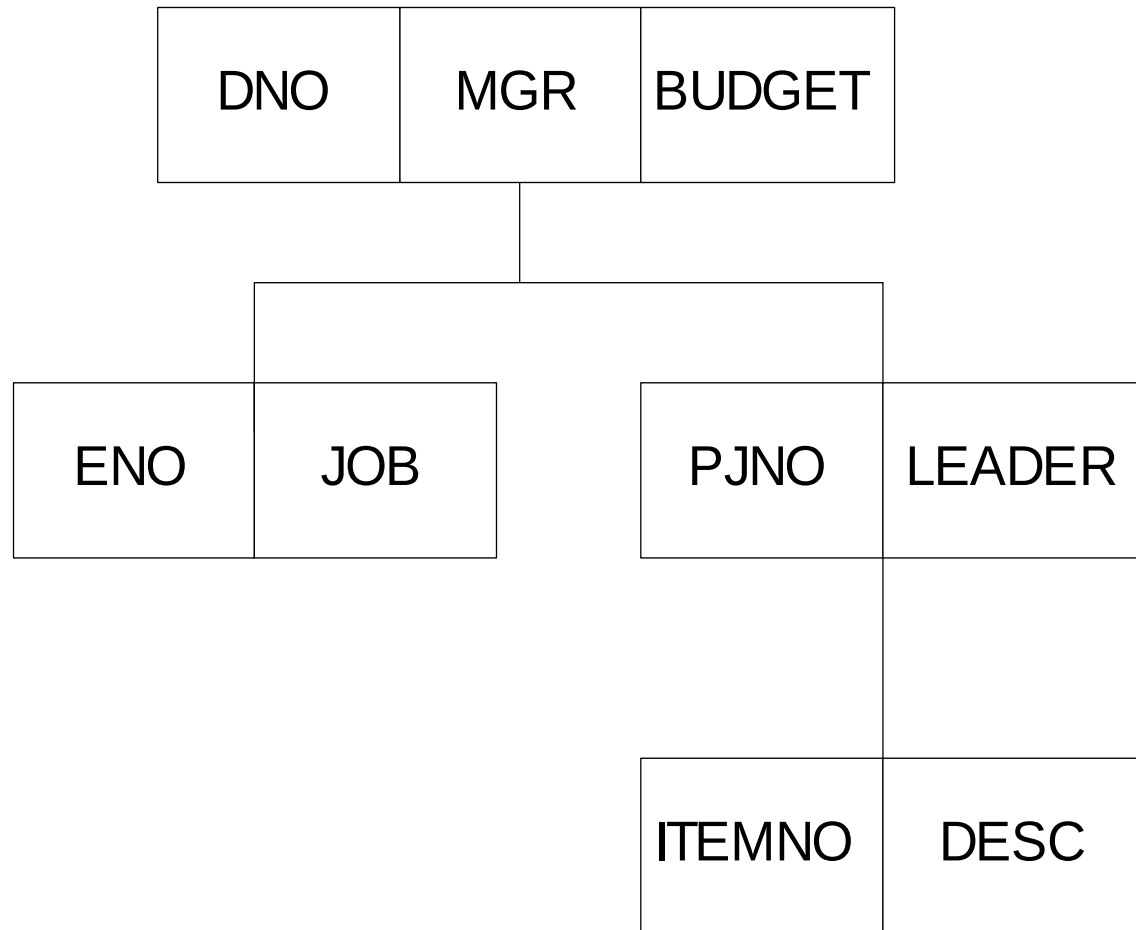
Translator Generator Approach of data conversion



DEFINE and CONVERT compile phase



As an example, consider the following hierarchical database:



Its DEFINE statements can be described in the following where for each DEFINE statement, a code is generated to allocate a new subtree in the internal buffer.

GROUP DEPT:

OCCURS FROM 1 TIMES;

FOLLOWED BY EOF;

PRECEDED BY HEX '01';

:

END EMP;

GROUP PROJ:

OCCURS FROM 0 TIMES;

PRECEDED BY HEX '03';

:

END PROJ;

2008/2/19
END DEPT;

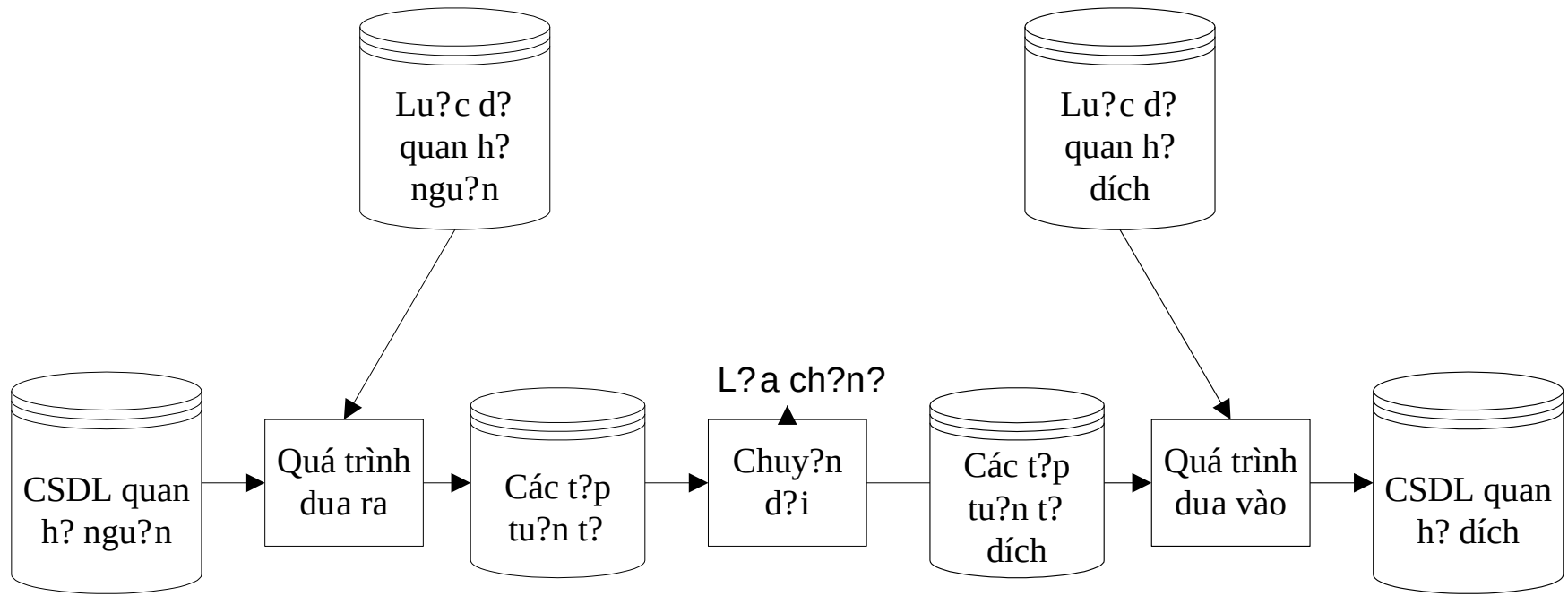
For each user-written CONVERT statement, we can produce a customized program. Take the DEPT FORM from the above DEFINE statement:

T1 = SELECT (FROM DEPT WHERE BUDGET GT '100');

will produce the following program:

```
/* PROCESS LOOP FOR T1 */  
DO WHILE (not end of file);  
  CALL GET (DEPT);  
  IF BUDGET > '100'  
    THEN CALL BUFFER_SWAP (T1, DEPT);  
END
```

Data conversion: Logical Level Translation Approach



System flow diagram for data conversion from source relational to target relational

Case study of converting a relational database from DB2 to Oracle

Business requirements

A company has two regions A and B.

- Each region forms its own departments.
- Each department approves many trips in a year.
- Each staff makes many trips in a year.
- In each trip, a staff needs to hire cars for transportation.
- Each hired car can carry many staff for each trip.
- A staff can be either a manager or an engineer.

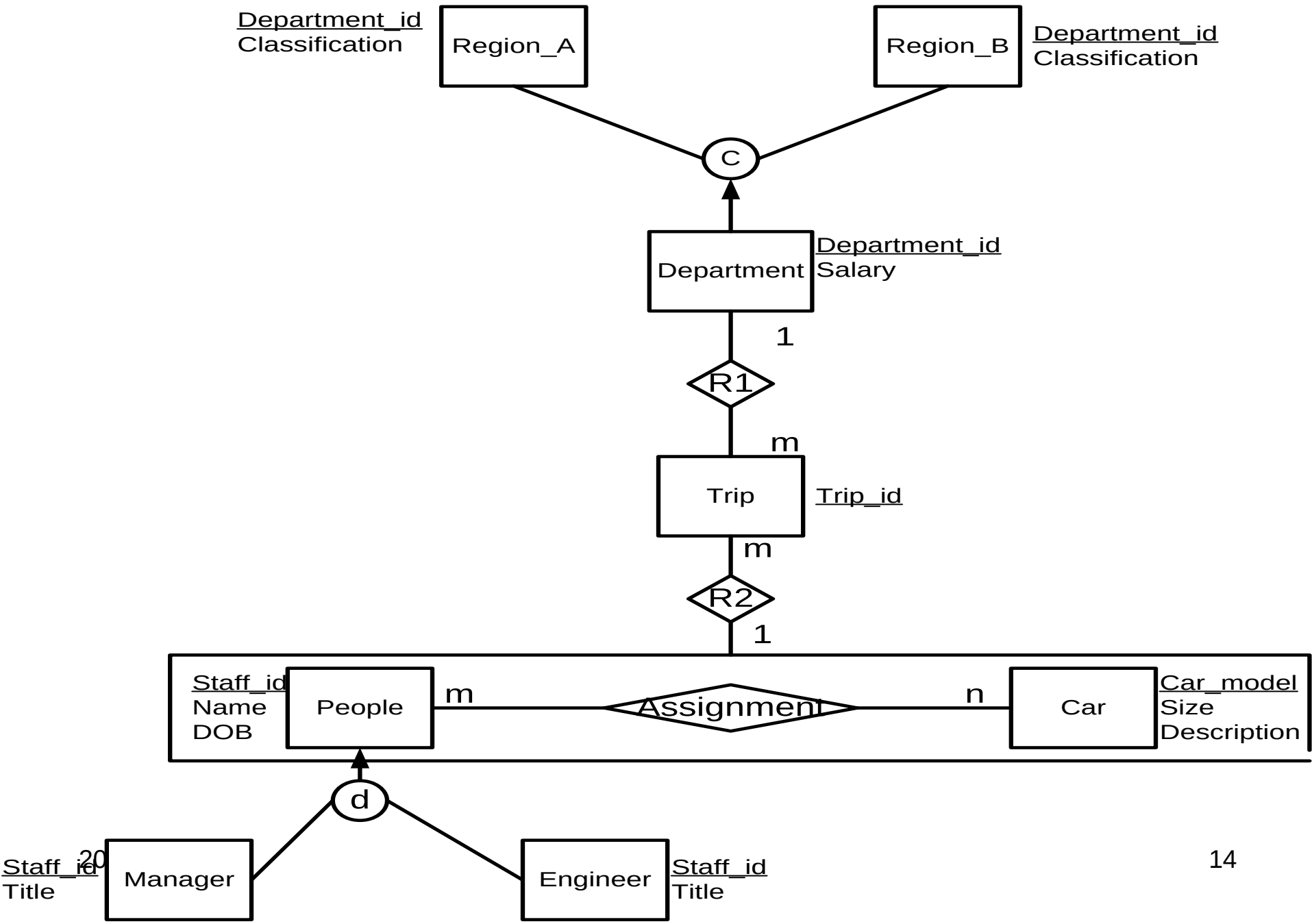
Data Requirements

Relation Department(*Department_id, Salary)
Relation Region_A (Department_id, Classification)
Relation Region_B (Department_id, Classification)
Relation Trip (Trip_id, *Car_model, *Staff_id, *Department_id)
Relation People (Staff_id, Name, DOB)
Relation Car (Car_model, Size, Description)
Relation Assignment(*Car_model, *Staff_id)
Relation Engineer (*Staff_id, Title)
Relation Manager (*Staff_id, Title)

ID: Department.Department_id $\subseteq$ (Region_A.Department_id $\cup$ Region_B.Department_id)
ID: Trip.Department_id $\subseteq$ Department.Department_id
ID: Trip.Car_model $\subseteq$ Assignment.Car_model
ID: Trip.Staff_id $\subseteq$ Assignment.Staff_id
ID: Assignment.Car_model $\subseteq$ Car.Car_model
ID: Assignment.Staff_id $\subseteq$ People.Staff_id
ID: Engineer.Staff_id $\subseteq$ People.Staff_id
ID: Manager.Staff_id $\subseteq$ People.Staff_id

Where ID = Inclusion Dependence, Underlined are primary keys and “*” prefixed are foreign keys.

Extended Entity Relationship Model for database Trip



Schema for target relational database Trip

Create Table Car

(CAR_MODEL character (10),
SIZE character (10),
DESCRIPT character (20),
STAFF_ID character (5),
primary key (CAR_MODEL))

Create Table Depart

(DEP_ID character (5),
SALARY numeric (8),
primary key (DEP_ID))

Create Table People

(STAFF_ID character (4),
NAME character (20),
DOB datetime,
primary key (STAFF_ID))

Create Table Reg_A

(DEP_ID character (5),
DESCRIP character (20),
primary key (DEP_ID))

Create Table Reg_B

(DEP_ID character (5),
DESCRIPT character (20),
primary key (DEP_ID))

Schema for target relational database Trip (continue)

Create Table trip

(TRIP_ID character (5),
primary key (TRIP_ID))

Create Table Engineer

(TITLE character (20),
STAFF_ID character (4),
Foreign Key (STAFF_ID) REFERENCES People(STAFF_ID),
Primary key (STAFF_ID))

Create Table Manager

(TITLE character (20),
STAFF_ID character (4),
Foreign Key (STAFF_ID) REFERENCES People(STAFF_ID),
Primary key (STAFF_ID))

Create Table Assign

(CAR_MODEL character (10),
Foreign Key (CAR_MODEL) REFERENCES Car(CAR_MODEL),
STAFF_ID character (4),
Foreign Key (STAFF_ID) REFERENCES People(STAFF_ID),
primary key (CAR_MODEL,STAFF_ID))

ALTER TABLE trip ADD CAR_MODEL character (10) null

ALTER TABLE trip ADD STAFF_ID character (4) null

2008/2/19
ALTER TABLE trip ADD Foreign Key (CAR_MODEL,STAFF_ID) REFERENCES Assign(CAR_MODEL,STAFF_ID)

Data insertion for target relational database Trip

```

INSERT INTO trip_ora.CAR (CAR_MODEL,CAR_SIZE,DESCRIPT,STAFF_ID) VALUES ('DA-02      ', '165      ', 'Long car      ', 'A001 ')
INSERT INTO trip_ora.CAR (CAR_MODEL,CAR_SIZE,DESCRIPT,STAFF_ID) VALUES ('MZ-18      ', '120      ', 'Small sportics ', 'B004 ')
INSERT INTO trip_ora.CAR (CAR_MODEL,CAR_SIZE,DESCRIPT,STAFF_ID) VALUES ('R-023      ', '150      ', 'Long car      ', 'A002 ')
INSERT INTO trip_ora.CAR (CAR_MODEL,CAR_SIZE,DESCRIPT,STAFF_ID) VALUES ('SA-38      ', '120      ', 'New dark blue   ', 'D001 ')
INSERT INTO trip_ora.CAR (CAR_MODEL,CAR_SIZE,DESCRIPT,STAFF_ID) VALUES ('WZ-01      ', '1445     ', 'Middle Sportics ', 'B004 ')
INSERT INTO trip_ora.DEPART (DEP_ID,SALARY) VALUES ('AA001', 35670)
INSERT INTO trip_ora.DEPART (DEP_ID,SALARY) VALUES ('AB001', 30010)
INSERT INTO trip_ora.DEPART (DEP_ID,SALARY) VALUES ('BA001', 22500)
INSERT INTO trip_ora.DEPART (DEP_ID,SALARY) VALUES ('BB001', 21500)
INSERT INTO trip_ora.PEOPLE (STAFF_ID,NAME,DOB) VALUES ('A001', 'Alexender      ', '7-1-1962')
INSERT INTO trip_ora.PEOPLE (STAFF_ID,NAME,DOB) VALUES ('A002', 'April          ', '5-24-1975')
INSERT INTO trip_ora.PEOPLE (STAFF_ID,NAME,DOB) VALUES ('B001', 'Bobby          ', '12-5-1987')
INSERT INTO trip_ora.PEOPLE (STAFF_ID,NAME,DOB) VALUES ('B002', 'Bladder        ', '1-3-1980')
INSERT INTO trip_ora.PEOPLE (STAFF_ID,NAME,DOB) VALUES ('B003', 'Brent          ', '12-15-1979')
INSERT INTO trip_ora.PEOPLE (STAFF_ID,NAME,DOB) VALUES ('B004', 'Brelendar      ', '8-18-1963')
INSERT INTO trip_ora.PEOPLE (STAFF_ID,NAME,DOB) VALUES ('C001', 'Calvin         ', '4-3-1977')
INSERT INTO trip_ora.PEOPLE (STAFF_ID,NAME,DOB) VALUES ('C002', 'Cheven         ', '2-2-1974')
INSERT INTO trip_ora.PEOPLE (STAFF_ID,NAME,DOB) VALUES ('C003', 'Clevarance     ', '12-6-1987')
INSERT INTO trip_ora.PEOPLE (STAFF_ID,NAME,DOB) VALUES ('D001', 'Dave           ', '8-17-1964')
INSERT INTO trip_ora.PEOPLE (STAFF_ID,NAME,DOB) VALUES ('D002', 'Davis          ', '3-19-1988')
INSERT INTO trip_ora.PEOPLE (STAFF_ID,NAME,DOB) VALUES ('D003', 'Denny          ', '8-5-1985')
INSERT INTO trip_ora.PEOPLE (STAFF_ID,NAME,DOB) VALUES ('D004', 'Denny          ', '2-21-1998')
INSERT INTO trip_ora.REG_A (DEP_ID,DESCRIP) VALUES ('AA001', 'Class A Manager ')
INSERT INTO trip_ora.REG_A (DEP_ID,DESCRIP) VALUES ('AB001', 'Class A Manager ')
INSERT INTO trip_ora.REG_A (DEP_ID,DESCRIP) VALUES ('AC001', 'Class A Manager ')
INSERT INTO trip_ora.REG_A (DEP_ID,DESCRIP) VALUES ('AD001', 'Class A Assistnat ')
INSERT INTO trip_ora.REG_A (DEP_ID,DESCRIP) VALUES ('AE001', 'Class A Assistnat ')
INSERT INTO trip_ora.REG_A (DEP_ID,DESCRIP) VALUES ('AF001', 'Class A Assistnat ')
INSERT INTO trip_ora.REG_A (DEP_ID,DESCRIP) VALUES ('AG001', 'Class A Clark    ')
INSERT INTO trip_ora.REG_A (DEP_ID,DESCRIP) VALUES ('AH001', 'Class A Assistant ')
INSERT INTO trip_ora.REG_B (DEP_ID,DESCRIPT) VALUES ('BA001', 'Class B Manager ')
INSERT INTO trip_ora.REG_B (DEP_ID,DESCRIPT) VALUES ('BB001', 'Class B Manager ')
INSERT INTO trip_ora.REG_B (DEP_ID,DESCRIPT) VALUES ('BC001', 'Class B Manager ')
INSERT INTO trip_ora.REG_B (DEP_ID,DESCRIPT) VALUES ('BD001', 'Class B Assistant ')
INSERT INTO trip_ora.REG_B (DEP_ID,DESCRIPT) VALUES ('BE001', 'Class B Assistant ')

```

Data insertion for target relational database Trip (continue)

```
INSERT INTO trip_ora.REG_B (DEP_ID,DESCRIPT) VALUES ('BF001', 'Class B Assistant  ')
INSERT INTO trip_ora.REG_B (DEP_ID,DESCRIPT) VALUES ('BG001', 'Class B Clark    ')
INSERT INTO trip_ora.REG_B (DEP_ID,DESCRIPT) VALUES ('BH001', 'Class B Assistant  ')

INSERT INTO trip_ora.TRIP (TRIP_ID) VALUES ('T0001')
INSERT INTO trip_ora.TRIP (TRIP_ID) VALUES ('T0002')
INSERT INTO trip_ora.TRIP (TRIP_ID) VALUES ('T0003')
INSERT INTO trip_ora.TRIP (TRIP_ID) VALUES ('T0004')

INSERT INTO trip_ora.ENGINEER (TITLE,STAFF_ID) VALUES ('Electronic Engineer ', 'A001')
INSERT INTO trip_ora.ENGINEER (TITLE,STAFF_ID) VALUES ('Senior Engineer   ', 'B003')
INSERT INTO trip_ora.ENGINEER (TITLE,STAFF_ID) VALUES ('Electronic Engineer ', 'B004')
INSERT INTO trip_ora.ENGINEER (TITLE,STAFF_ID) VALUES ('Junior Engineer    ', 'C002')
INSERT INTO trip_ora.ENGINEER (TITLE,STAFF_ID) VALUES ('System Engineer    ', 'D001')
INSERT INTO trip_ora.ENGINEER (TITLE,STAFF_ID) VALUES ('Material Engineer   ', 'D002')
INSERT INTO trip_ora.ENGINEER (TITLE,STAFF_ID) VALUES ('Parts Engineer     ', 'D003')
INSERT INTO trip_ora.MANAGER (TITLE,STAFF_ID) VALUES ('Sales Manager      ', 'A002')
INSERT INTO trip_ora.MANAGER (TITLE,STAFF_ID) VALUES ('Marketing Manager   ', 'B001')
INSERT INTO trip_ora.MANAGER (TITLE,STAFF_ID) VALUES ('Sales Manager       ', 'B002')
INSERT INTO trip_ora.MANAGER (TITLE,STAFF_ID) VALUES ('General Manager     ', 'C001')
INSERT INTO trip_ora.MANAGER (TITLE,STAFF_ID) VALUES ('Marketing Manager    ', 'C003')
INSERT INTO trip_ora.ASSIGN (CAR_MODEL,STAFF_ID) VALUES ('MZ-18      ', 'A002')
INSERT INTO trip_ora.ASSIGN (CAR_MODEL,STAFF_ID) VALUES ('MZ-18      ', 'B001')
INSERT INTO trip_ora.ASSIGN (CAR_MODEL,STAFF_ID) VALUES ('MZ-18      ', 'D003')
INSERT INTO trip_ora.ASSIGN (CAR_MODEL,STAFF_ID) VALUES ('R-023      ', 'B004')
INSERT INTO trip_ora.ASSIGN (CAR_MODEL,STAFF_ID) VALUES ('R-023      ', 'C001')
INSERT INTO trip_ora.ASSIGN (CAR_MODEL,STAFF_ID) VALUES ('R-023      ', 'D001')
INSERT INTO trip_ora.ASSIGN (CAR_MODEL,STAFF_ID) VALUES ('SA-38      ', 'A001')
INSERT INTO trip_ora.ASSIGN (CAR_MODEL,STAFF_ID) VALUES ('SA-38      ', 'A002')
INSERT INTO trip_ora.ASSIGN (CAR_MODEL,STAFF_ID) VALUES ('SA-38      ', 'D002')
INSERT INTO trip_ora.ASSIGN (CAR_MODEL,STAFF_ID) VALUES ('WZ-01      ', 'B002')
INSERT INTO trip_ora.ASSIGN (CAR_MODEL,STAFF_ID) VALUES ('WZ-01      ', 'B003')
INSERT INTO trip_ora.ASSIGN (CAR_MODEL,STAFF_ID) VALUES ('WZ-01      ', 'C002')
```

Data insertion for target relational database Trip (continue)

```
UPDATE trip_ora52.TRIP
SET DEPT_ID= 'AA001'
, CAR_MODEL= 'MZ-18  '
, STAFF_ID= 'A002'
, CAR_MODEL= 'MZ-18  '
, STAFF_ID= 'A002'
WHERE TRIP_ID='T0001'
```

```
UPDATE trip_ora52.TRIP
SET DEPT_ID= 'AA001'
, CAR_MODEL= 'MZ-18  '
, STAFF_ID= 'B001'
, CAR_MODEL= 'MZ-18  '
, STAFF_ID= 'B001'
WHERE TRIP_ID='T0002'
```

```
UPDATE trip_ora52.TRIP
SET DEPT_ID= 'AB001'
, CAR_MODEL= 'SA-38  '
, STAFF_ID= 'A002'
, CAR_MODEL= 'SA-38  '
, STAFF_ID= 'A002'
WHERE TRIP_ID='T0003'
```

```
UPDATE trip_ora52.TRIP
SET DEPT_ID= 'BB001'
, CAR_MODEL= 'SA-38  '
, STAFF_ID= 'D002'
, CAR_MODEL= 'SA-38  '
, STAFF_ID= 'D002'
WHERE TRIP_ID='T0004'
```

Data Integration

Step 1: Merge by Union

Relation Ra

<u>A₁</u>	A ₂
a ₁₁	a ₂₁
a ₁₂	a ₂₂

Relation Rb

<u>A₁</u>	A ₃
a ₁₃	a ₃₁
a ₁₄	a ₃₂

==>

Relation Rx

<u>A₁</u>	A ₂	A ₃
a ₁₁	a ₂₁	null
a ₁₂	a ₂₂	null
a ₁₃	null	a ₃₁
a ₁₄	null	a ₃₂

Step 2: Merge classes by generalization

Relation Ra

<u>A₁</u>	A ₂
a ₁₁	a ₂₁
a ₁₂	a ₂₂

Relation Rx1

<u>A₁</u>	A ₂
a ₁₁	a ₂₁
a ₁₂	a ₂₂

Relation Rx

<u>A₁</u>	A ₂	A ₃
a ₁₁	a ₂₁	null
a ₁₂	a ₂₂	null
a ₁₃	null	a ₃₁
a ₁₄	null	a ₃₂

==>

Relation Rb

<u>A₁</u>	A ₃
a ₁₃	a ₃₁
a ₁₄	a ₃₂

Relation Rx2

<u>A₁</u>	A ₃
a ₁₃	a ₃₁
a ₁₄	a ₃₂

Step 3: Merge classes by inheritance

Relation Rb

<u>A₁</u>	A ₂
a ₁₁	a ₂₁
a ₁₂	a ₂₂

Relation Rxb

<u>A₁</u>	A ₂	A ₃
a ₁₁	a ₂₁	a ₃₁
a ₁₂	a ₂₂	null

=>

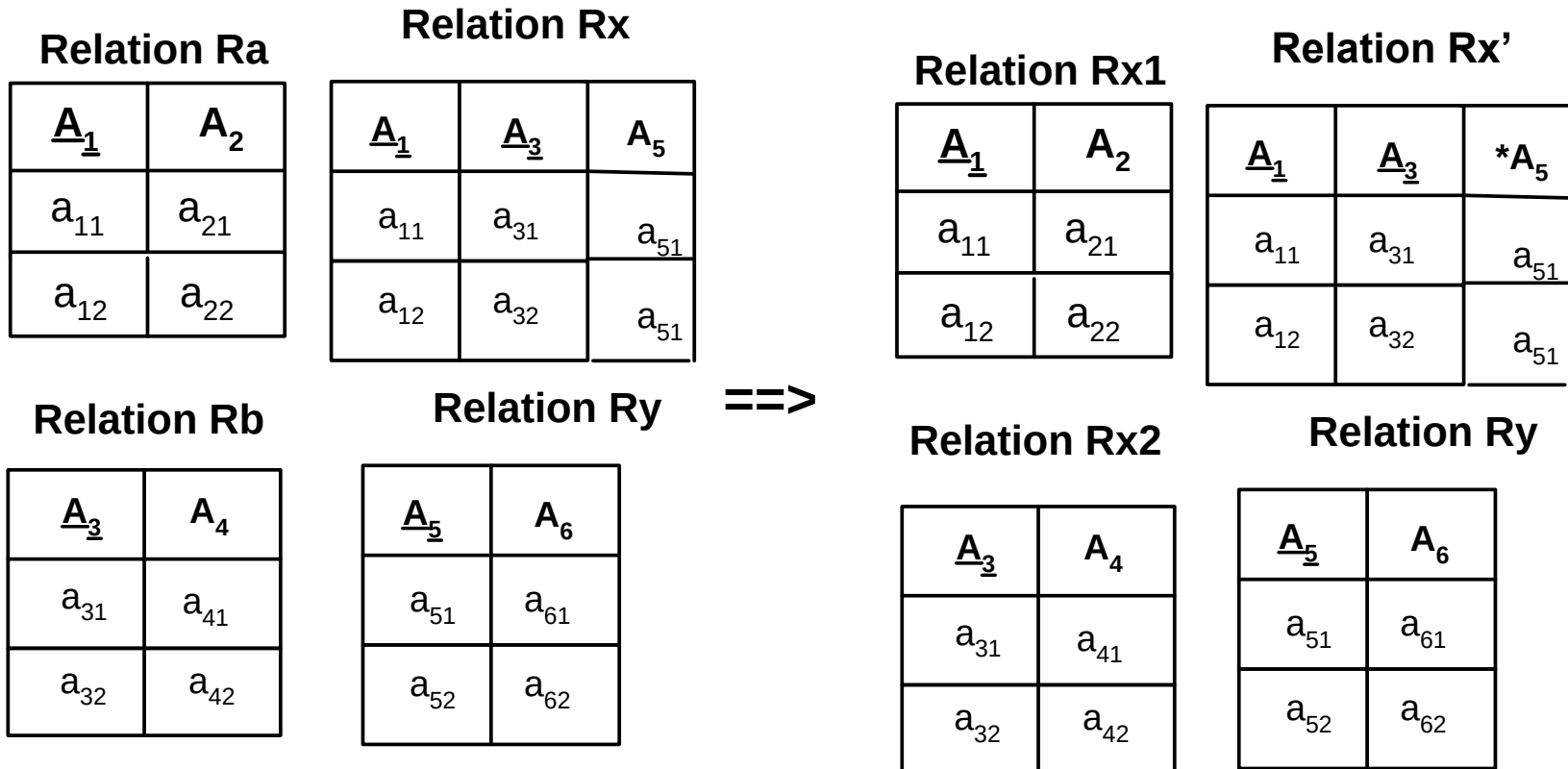
Relation Ra

<u>A₁</u>	A ₃
a ₁₁	a ₃₁

Relation Rxa

<u>A₁</u>	A ₃
a ₁₁	a ₃₁

Step 4 Merge classes by aggregation



Step 5 Merge classes by categorization

Relation Ra

<u>A₁</u>	A ₂
a ₁₁	a ₂₁
a ₁₂	a ₂₂

Relation Rb

<u>A₁</u>	A ₃
a ₁₃	a ₃₁
a ₁₄	a ₃₂

==>

Relation Rx

<u>A₁</u>	<u>A₄</u>
a ₁₁	a ₄₁
a ₁₂	a ₄₂
a ₁₃	a ₄₃
a ₁₄	a ₄₄

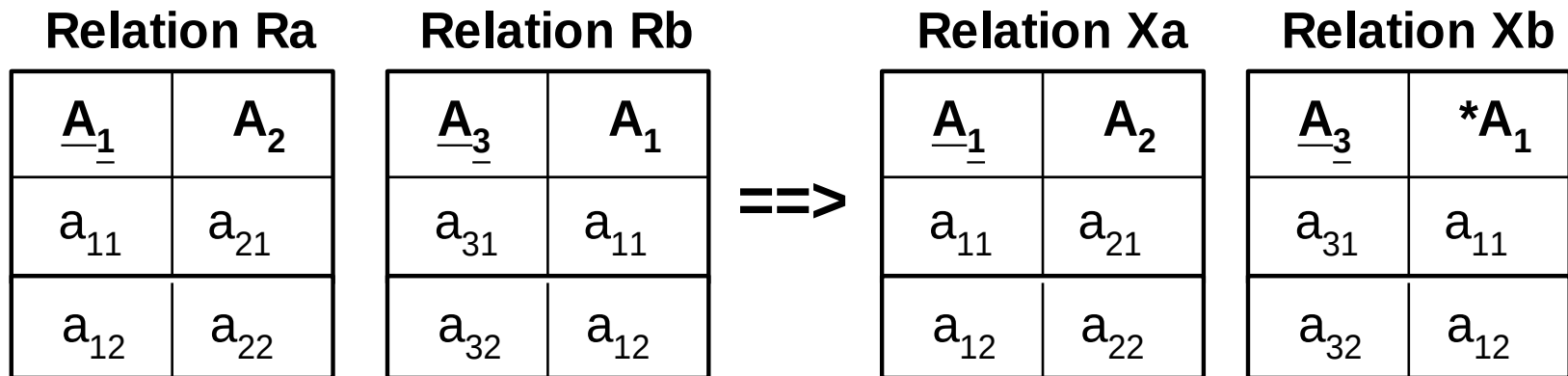
Relation Ra

<u>A₁</u>	A ₂
a ₁₁	a ₂₁
a ₁₂	a ₂₂

Relation Rb

<u>A₁</u>	A ₃
a ₁₃	a ₃₁
a ₁₄	a ₃₂

Step 6 Merge classes by implied relationship



Step 7 Merge relationship by subtype

Relation Ra

<u>A₁</u>	A ₂
a ₁₁	a ₂₁
a ₁₂	a ₂₂

Relation Rb

<u>A₃</u>	*A ₁
a ₃₁	a ₁₁
a ₃₂	a ₁₂

Relation Xa

<u>A₁</u>	A ₂
a ₁₁	a ₂₁
a ₁₂	a ₂₂

Relation Xc

* <u>A₃</u>	*A ₁
a ₃₁	a ₁₁
a ₃₂	a ₁₂

Relation Ra'

<u>A₁</u>	A ₂
a ₁₁	a ₂₁
a ₁₂	a ₂₂

Relation Rb'

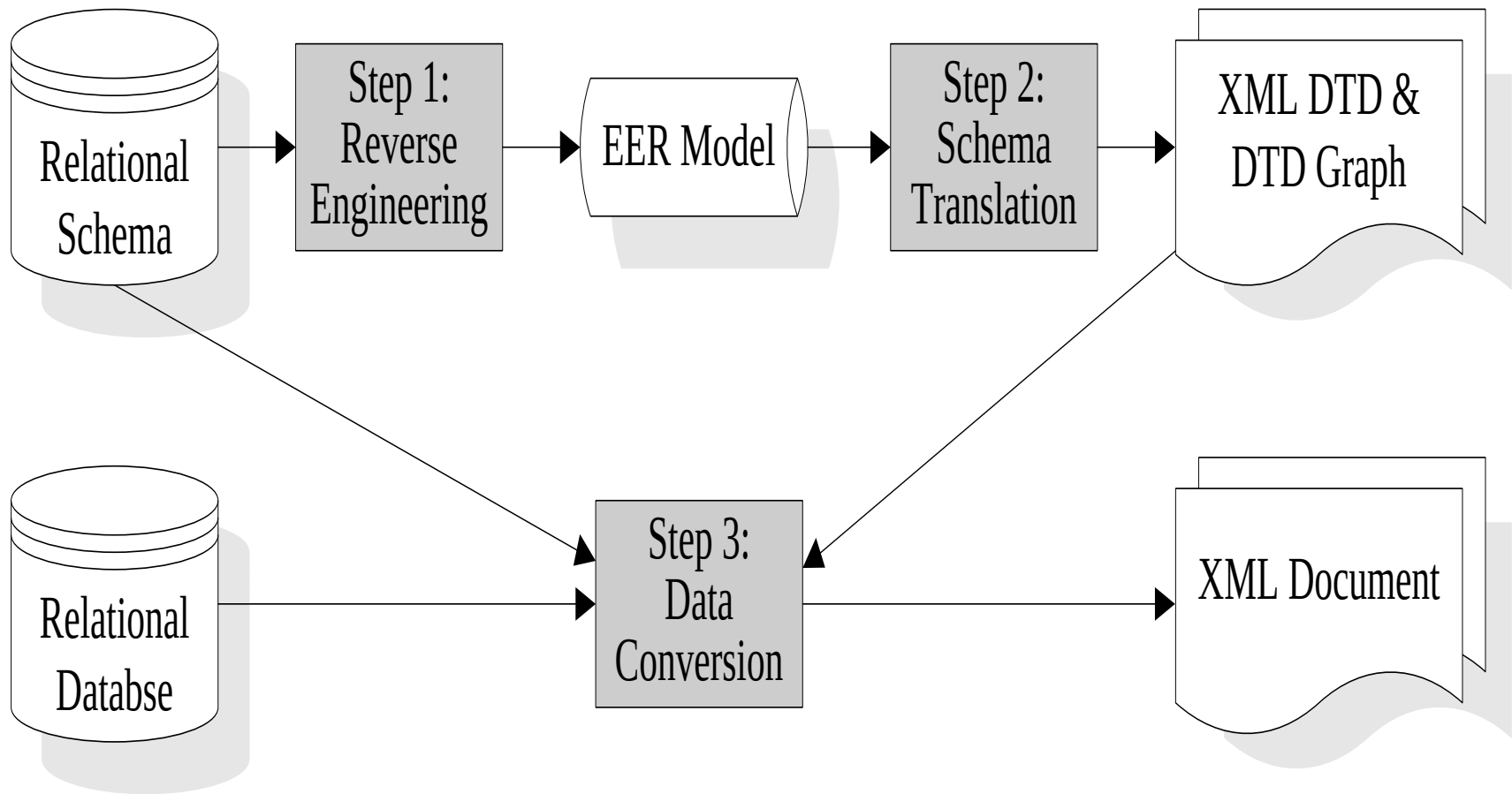
<u>A₃</u>	A ₁
a ₃₁	a ₁₁
a ₃₃	null

==>

Relation Xb

<u>A₃</u>	A ₁
a ₃₁	a ₁₁
a ₃₂	a ₁₂
a ₃₃	null

Data conversion from relational into XML



Architecture of Re-engineering Relational Database into XML Document

Methodology of converting RDB into XML

As the result of the schema translation, we translate an EER model into different views of XML schemas based on their selected root elements. For each translated XML schema, we can read its corresponding source relation sequentially by embedded SQL starting a parent relation. The tuple can then be loaded into XML document according to the mapped XML DTD. Then we read the corresponding child relation tuple(s), and load them into XML document.

Algorithm of transforming RDB into XML

```
begin
  while not end of element do
    read an element from the translated target DTD;
    read the tuple of a corresponding relation of the element from the source relational database;
    load this tuple into a target XML document;
    read the child elements of the element according to the DTD;
    while not at end of the corresponding child relation in the source relational database do
      read the tuple from the child relation such that the child's corresponding to the processed
        parent relation's tuple;
      load the tuple to the target XML document;
    end loop // end inner loop
  end loop // end outer loop
end
```

Step 1: Reverse Engineering Relational Schema into an EER Model

By use of classification tables to define the relationship between keys and attributes in all relations, we can recover their data semantics in the form of an EER model.

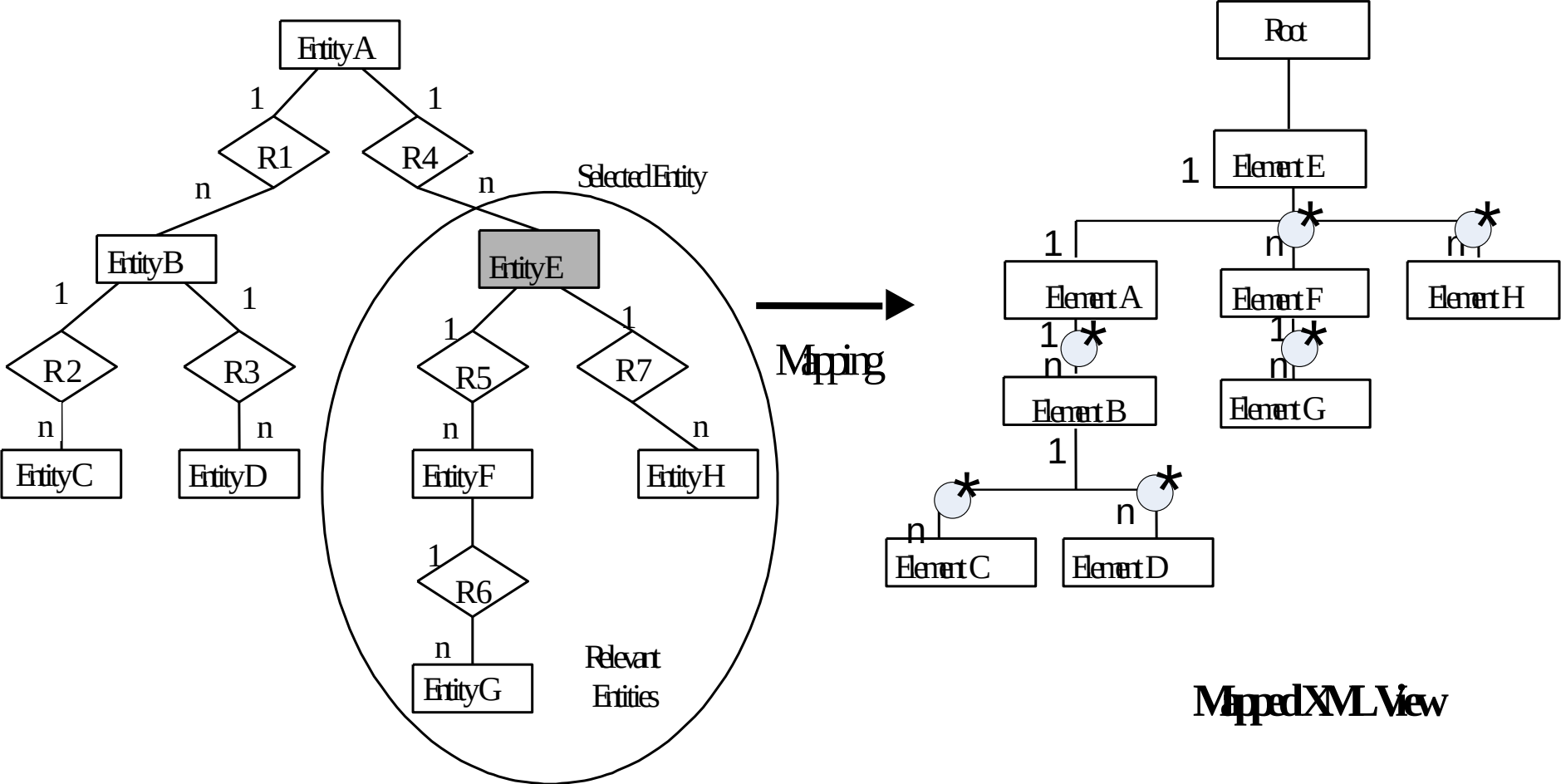
Step 2: Data Conversion from Relational into XML document

We can map the data semantics in the EER model into DTD-graph according to their data dependencies constraints. These constraints can then be transformed into DTD as XML schema.

Step 2.1 Defining a Root Element

To select a root element, we must put its relevant information into an XML schema. Relevance concerns with the entities that are related to a selected entity by the user. The relevant classes include the selected entity and all its relevant entities that are navigable.

In an EER mode, we can navigate from entity to another entity in correspondence to XML hierarchical containment tree model.



FER Model

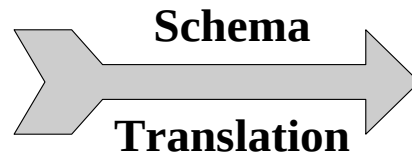
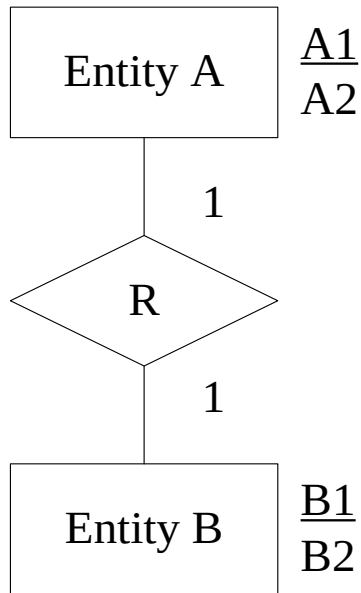
Mapped XML View

Step 2.1 Mapping Cardinality from RDB to XML

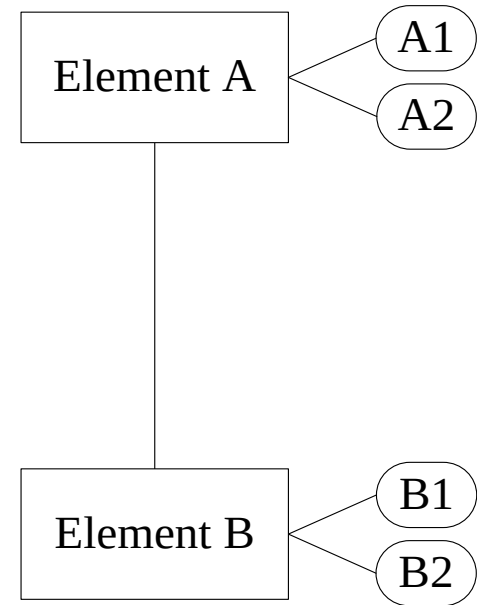
In DTD, we translate one-to-one cardinality into parent and child element and one-to-many cardinality into parent and child element with multiple occurrences. In many-to-many cardinality, it is mapped into DTD of a hierarchy structure with ID and IDREF.

One-to-one cardinality

EER Model



DTD Graph



Relational Schema

Relation A(A1, A2)
Relation B(B1, B2, *A1)

DTD

```
<!ELEMENT A(B)>  
<!!ATTLIST A A1 CDATA #REQUIRED>  
<!!ATTLIST A A2 CDATA #REQUIRED>  
<!ELEMENT B EMPTY>  
<!!ATTLIST B B1 CDATA #REQUIRED>  
<!!ATTLIST B B2 CDATA #REQUIRED>
```

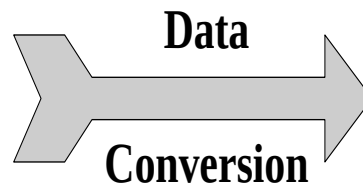
One-to-one cardinality

Relation A

<u>A1</u>	A2
a11	a21
a12	a22

Relation B

<u>B1</u>	B2	*A1
b11	b21	a11
b12	b22	a12



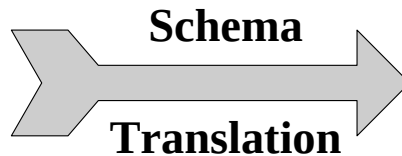
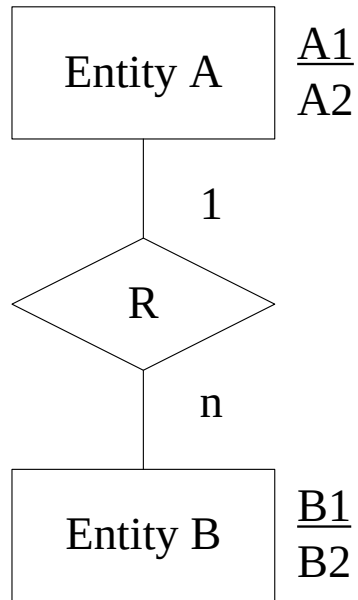
XML Document

```
<A A1="a11" A2="a21">  
  <B B1="b11" B2="b21"></B>  
</A>
```

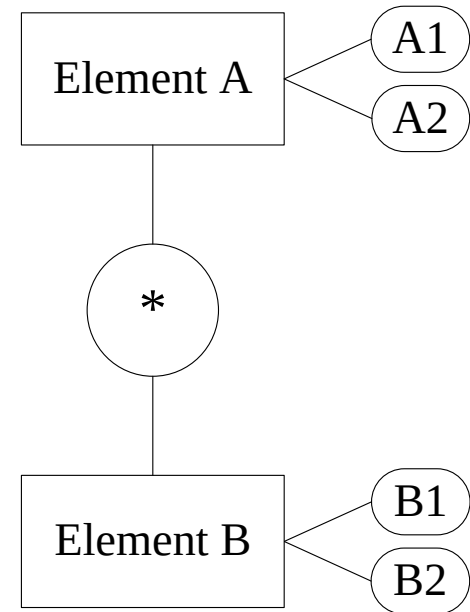
```
<A A1="a12" A2="a22">  
  <B B1="b12" B2="b22"></B>  
</A>
```

One-to-many cardinality

EER Model



DTD Graph



Relational Schema

Relation A(A1, A2)
Relation B(B1, B2, *A1)

DTD

```
<!ELEMENT A(B)*>  
<!ATTLIST A A1 CDATA #REQUIRED>  
<!ATTLIST A A2 CDATA #REQUIRED>  
<!ELEMENT B EMPTY>  
<!ATTLIST B B1 CDATA #REQUIRED>  
<!ATTLIST B B2 CDATA #REQUIRED>
```

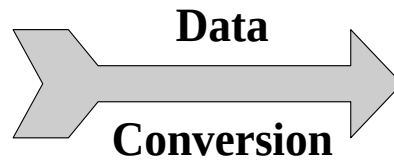
One-to-many cardinality

Relation A

<u>A1</u>	A2
a11	a21
a12	a22

Relation B

<u>B1</u>	B2	*A1
b11	b21	a11
b12	b22	a12
b13	b23	a12



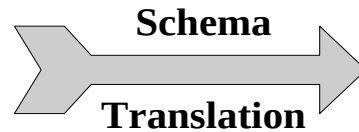
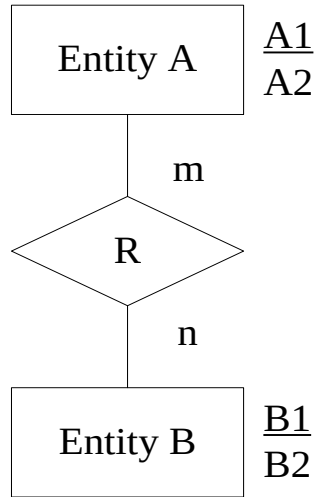
XML Document

```
<A A1="a11" A2="a21">  
  <B B1="b11" B2="b21"></B>  
</A>
```

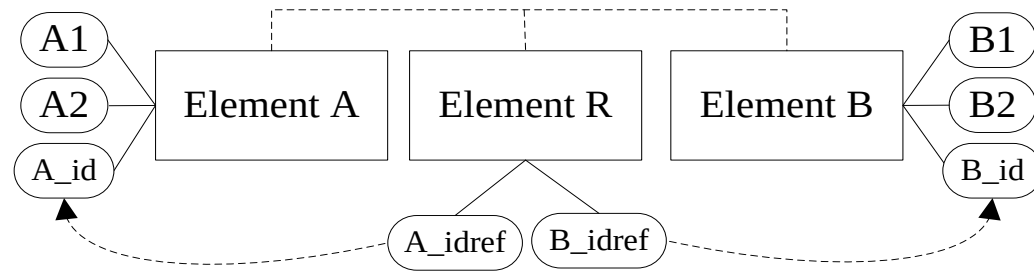
```
<A A1="a12" A2="a22">  
  <B B1="b12" B2="b22"></B>  
  <B B1="b13" B2="b23"></B>  
</A>
```

Many-to-many cardinality

EER Model



DTD Graph



DTD

```
<!ELEMENT A EMPTY>
<!ATTLIST A A1 CDATA #REQUIRED>
<!ATTLIST A A2 CDATA #REQUIRED>
<!ATTLIST A A_id ID #REQUIRED>
<!ELEMENT R EMPTY>
<!ATTLIST R A_idref IDREF #REQUIRED>
<!ATTLIST R B_idref IDREF #REQUIRED>
<!ELEMENT B EMPTY>
<!ATTLIST B B1 CDATA #REQUIRED>
<!ATTLIST B B2 CDATA #REQUIRED>
<!ATTLIST B B_id ID #REQUIRED>
```

Relational Schema

Relation A(A1, A2)
Relation B(B1, B2)
Relation R(*A1, *B1)

Many-to-many cardinality

Relation A

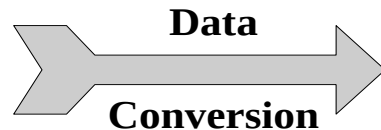
<u>A1</u>	A2
a11	a21
a12	a22

Relation B

<u>B1</u>	B2
b11	b21
b12	b22

Relation R

<u>*A1</u>	<u>*B1</u>
a11	b11
a12	b12
a11	b12



XML Document

```
<A A1="a11" A2="a21" A_id="1"></A>  
<B B1="b11" B2="b21" B_id="2"></B>  
<R A_idref="1" B_idref="2"></R>
```

```
<A A1="a12" A2="a22" A_id="3"></A>  
<B B1="b12" B2="b22" B_id="4"></B>  
<R A_id="3" B_idref="4"></R>
```

```
<R A_id="1" B_idref="4"></R>
```


Case Study

Consider a case study of a Hospital Database System. In this system, a patient can have many record folders. Each record folder can contain many different medical records of the patient. A country has many patients. Once a record folder is borrowed, a loan history is created to record the details about it.

Hospital Relational Schema

Relation Patient (HK_ID, Patient_Name)

Relation Record_Folder (Folder_No, Location, *HKID)

Relation Medical_Record (Medical_Rec_No,
Create_Date, Sub_Type, *Folder_No)

Relation Borrower (*Borrower_No, Borrower_Name)

Relation Borrow (*Borrower_No, *Folder_No)

Where underlined are primary keys, prefixed with “*”
are foreign keys

Relational Data

Patient table

E3766849

Smith

Record_Folder table

F_21

Hong Kong

E3766849

F_24

New Territories

E3766849

Borrower table

F_21

B1

F_21

B11

F_21

B21

F_21

B22

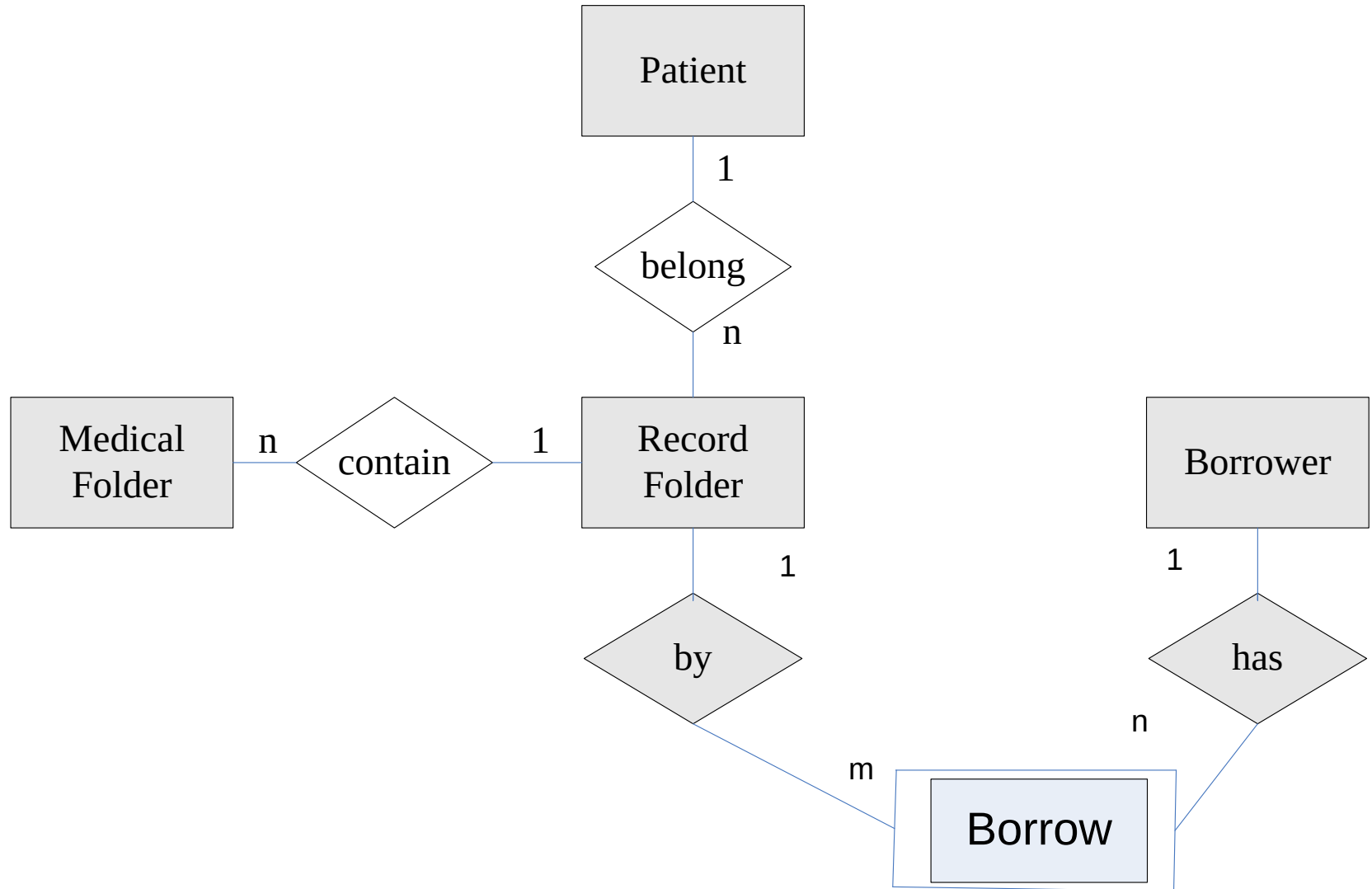
F_24

B22

Borrower_Name Table	<u>Borrower_no</u>	<u>Borrower_name</u>
	B1	Johnson
	B11	Choy
	B21	Fung
	B22	Lok

Medical_Record Table	<u>Medical_Rec_no</u>	<u>Create_Date</u>	<u>Sub_type</u>	<u>Folder_no</u>
	M_311999	Jan-01-1999	W	F_21
	M_322000	Nov-12-1998	W	F_21
	M_352001	Jan-15-2001	A	F_21
	M_362001	Feb-01-2001	A	F_21
	M_333333	Mar-03-2001	A	F_24

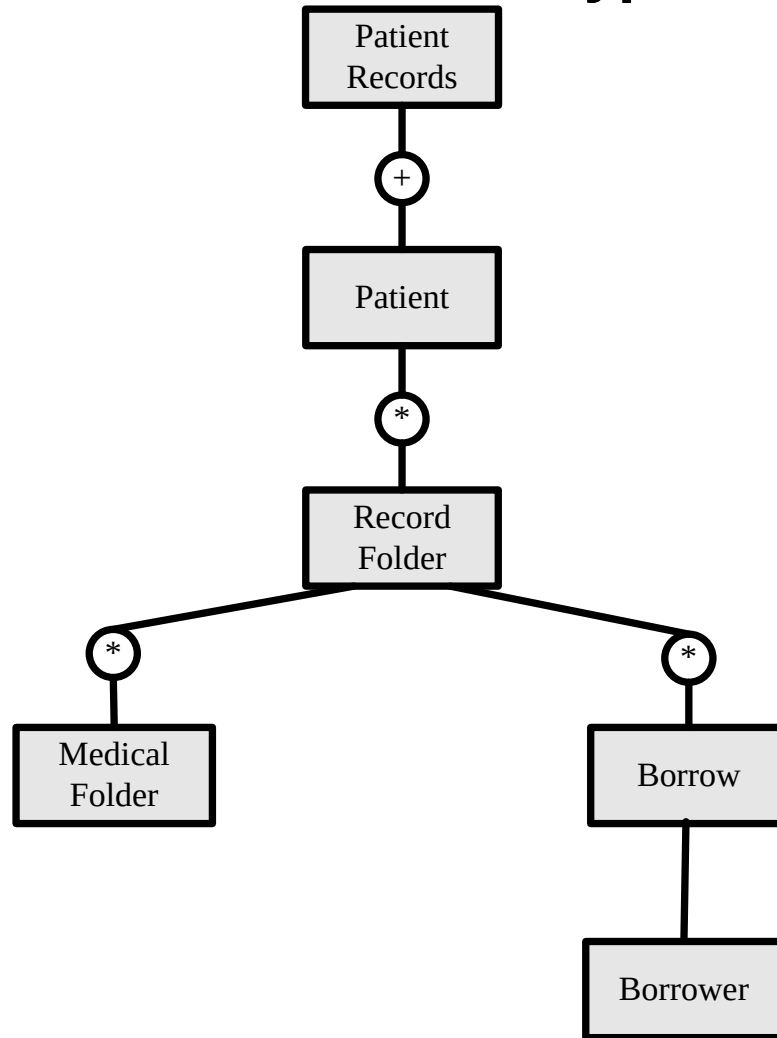
Step 1 Reverse engineer relational database into an EER model



Step 2 Translate EER model into DTD Graph and DTD

In this case study, suppose we concern the patient medical records, so the entity Patient is selected. Then we define a meaningful name for the root element, called Patient_Records. We start from the entity Patient in the EER model and then find the relevant entities for it. The relevant entities include the related entities that are navigable from the parent entity.

Translated Document Type Definition Graph



Translated Document Type Definition

<!ELEMENT Patient_Records (Patient+)>

<!ELEMENT Patient (Record_Folder*)>

<!ATTLIST Patient	HKID	CDATA	#REQUIRED>
-------------------	------	-------	------------

<!ATTLIST Patient	Patient_Name	CDATA	#REQUIRED>
-------------------	--------------	-------	------------

<!ELEMENT Record_Folder (Borrow*, Medical_Record*)>

<!ATTLIST Record_Folder	Folder_No	CDATA	#REQUIRED>
-------------------------	-----------	-------	------------

<!ATTLIST Record_Folder	Location	CDATA	#REQUIRED>
-------------------------	----------	-------	------------

<!ELEMENT Borrow (Borrower)>

<!ATTLIST Borrow	Borrower_No	CDATA	#REQUIRED>
------------------	-------------	-------	------------

<!ELEMENT Medical_Record EMPTY>

<!ATTLIST Medical_Record	Medical_Rec_No	CDATA	#REQUIRED>
--------------------------	----------------	-------	------------

<!ATTLIST Medical_Record	Create_Date	CDATA	#REQUIRED>
--------------------------	-------------	-------	------------

<!ATTLIST Medical_Record	Sub_Type	CDATA	#REQUIRED>
--------------------------	----------	-------	------------

<!ELEMENT Borrower EMPTY>

<!ATTLIST Borrower	Borrower_name	CDATA	#REQUIRED>
--------------------	---------------	-------	------------

Patient_Records>

<Patient Country_No="C0001" HKID="E3766849" Patient_Name="Smith">

<Record_Folder Folder_No="F_21" Location="Hong Kong">

<Borrow Borrower_No="B1">

<Borrower Borrower_name="Johnson" />

</Borrow>

<Borrow Borrower_No="B11">

<Borrower Borrower_name="Choy" />

</Borrow>

<Borrow Borrower_No="B21">

<Borrower Borrower_name="Fung" />

</Borrow>

<Borrow Borrower_No="B22">

<Borrower Borrower_name="Lok" />

</Borrow>

<Medical_Record Medical_Rec_No="M_311999" Create_Date="Jan-1-1999", Sub_Type="W"></Medical_Record>

<Medical_Record Medical_Rec_No="M_322000" Create_Date="Nov-12-1998", Sub_Type="W"></Medical_Record>

<Medical_Record Medical_Rec_No="M_352001" Create_Date="Jan-15-2001", Sub_Type="A"></Medical_Record>

<Medical_Record Medical_Rec_No="M_362001" Create_Date="Feb-01-2001", Sub_Type="A"></Medical_Record>

</Record_Folder>

<Record_Folder Folder_No="F_24" Location="New Territories">

<Borrow Borrower_No="B22">

<Borrower Borrower_name="Lok" />

</Borrow>

<Medical_Record Medical_Rec_No="M_333333" Create_Date="Mar-03-01", Sub_Type="A"></Medical_Record>

</Record_Folder>

</Patient>

Reading Assignment

Chapter 4 Data Conversion of “Information Systems Reengineering and Integration” by Joseph Fong, Springer Verlag, pp.160-198.

Lecture review question 6

How do you compare the pros and cons of using “Logical Level Translation Approach” with “Customized Program Approach” in data conversion?

CS5483 Tutorial Question 6

Convert the following relational database into an XML document:

Relation Car_rental

<u>Car_model</u>	<u>Staff_ID</u>	* <u>Trip_ID</u>
MZ-18	A002	T0001
MZ-18	B001	T0002
R-023	B004	T0001
R-023	C001	T0004
SA-38	A001	T0003
SA-38	A002	T0001

Relation Trip

<u>Trip_ID</u>	* <u>Department_ID</u>
T0001	AA001
T0002	AA001
T0003	AB001
T0004	BA001

Relation Department

<u>Department_ID</u>	Salary
AA001	35670
AB001	30010
BA001	22500